

The Traveling Flyerperson: An Introduction

Where do I place these flyers in Cambridge, MA?

Cambridge Developers
USA, Cambridge, MA

Abstract

We introduce *The Traveling Flyerperson*, a project that takes a simple civic question—where should a small software group place flyers in Cambridge, Massachusetts?—and uses it as a lens for exploring algorithms, geography, and decision-making. This primer presents the core ideas: the routing problem as a Traveling Salesman Problem, the legal and practical constraints of flyer-ing in Cambridge, and the interactive web application that ties them together. We leave the full statistical framework (survival analysis, point processes, Bayesian hierarchical models) to the companion paper and focus here on motivation, setup, and what makes this problem interesting.

Keywords

Traveling Salesman Problem, civic computing, Cambridge MA, route optimization

1 The Problem

You have a stack of flyers for a local software group. You live in Cambridge, Massachusetts—6.4 square miles of dense sidewalks, two research universities, and a long tradition of community bulletin boards. You want to post these flyers where people will actually see them, and you want to walk (or bike) an efficient route to do it.

This is a real problem we faced at cambridge-dev.org. It turns out to be more interesting than it looks.

1.1 Why Not Just Walk?

The naive approach—grab a stack, walk to familiar spots—works. But it raises questions you can’t answer without thinking harder:

- Are you visiting the *best* locations, or just the ones you know?
- Is your route efficient, or are you backtracking?
- How long do flyers actually last before someone removes them?
- Are there legal constraints you’re violating without knowing?

Each of these questions opens a door to a different area of computer science or statistics. The project walks through those doors one at a time.

2 The Routing Problem

Given n locations to visit, starting and ending at the Cambridge Public Library, what’s the shortest route? This is the Traveling Salesman Problem (TSP)—one of the most studied problems in combinatorial optimization.

For our purposes, the “distance” between two locations is the straight-line (Haversine) distance on the Earth’s surface. A tour is a permutation π of the locations with total cost:

$$\mathcal{C}(\pi) = \sum_{i=1}^{n-1} d(\pi_i, \pi_{i+1}) + d(\pi_n, \pi_1) \quad (1)$$

where $d(\cdot, \cdot)$ is the Haversine distance. We want the permutation that minimizes this cost.

2.1 Three Approaches

The application implements three solvers, each illustrating a different algorithmic strategy:

Brute Force. Try every possible ordering. There are $(n-1)!/2$ distinct tours. For our ~ 10 locations this is about 181,000 tours—feasible, but the approach collapses for larger inputs. This gives the *exact* optimal solution.

Nearest Neighbor. A greedy heuristic: from your current location, always go to the closest unvisited stop. Fast ($O(n^2)$) but often produces routes 20–30% longer than optimal because it doesn’t plan ahead.

2-Opt. Start with the Nearest Neighbor solution, then iteratively improve it: pick two edges in the tour, reverse the segment between them, and keep the change if it shortens the route. Repeat until no improvement is found. This is a *local search* method—it can’t guarantee the global optimum, but in practice it gets close.

3 The Constraints

Cambridge isn’t a blank grid. The problem has real-world constraints that make it more than a textbook exercise.

3.1 Legal Rules

Massachusetts law and Cambridge municipal code prohibit posting on:

- Public utility poles and lamp posts
- Shade trees and public buildings
- Sidewalks and public infrastructure

The fine is \$300 per violation. This immediately eliminates the most visible, highest-traffic locations—the ones a naive flyerperson would target first.

3.2 Permission and Policing

Even on private property, most locations require permission. Cafes curate their boards. Libraries have posting policies. MIT reserves campus boards for recognized student groups. Business Improvement Districts (like Central Square) employ crews that remove unauthorized postings daily.



Figure 1: Desktop view. The sidebar exposes solver selection, route summary, legal rules, and toggleable map layers. The map shows the computed tour over Cambridge with CARTO dark tiles.

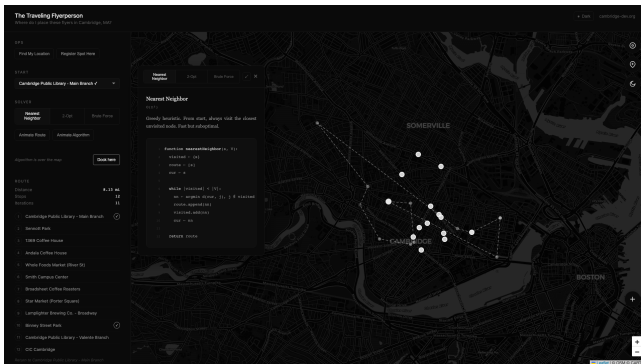


Figure 2: The algorithm panel expanded over the map. Pseudocode is rendered with line numbers; the active line highlights as the route is built step by step, mirroring a debugger's stepping behavior.

The net effect: the set of *valid* flying locations is much smaller than the set of *visible* ones, and survival time varies enormously by location type.

3.3 Geography

Cambridge is bounded by the Charles River to the south, Somerville to the north, and Boston to the east. The road network, one-way streets, and pedestrian paths constrain actual walking routes in ways that straight-line distance doesn't capture. (A future version of this project will incorporate real walking distances via OSRM or GraphHopper.)

4 The Application

The deployed application at letters-cambridge.web.app/stories-survival/ is a static SvelteKit site that runs entirely in the browser. No backend, no accounts, no data collection.

Key features: (1) Three solvers with animated, step-by-step execution showing pseudocode alongside the map (2) Evidence registry: photographs of placed flyers, geotagged via



Figure 3: Mobile layout with bottom sheet and floating action buttons for field use.

EXIF metadata (3) OpenStreetMap layers: traffic signals, bulletin board candidates (cafes, libraries, community centers), and transit stops (4) Legal overlay: visual indication of which locations are legally constrained (5) Mobile-first: swipeable bottom sheet, GPS integration, and floating action buttons for field use

The algorithm visualization is the application's primary pedagogical feature. Rather than treating the solver as a black box that returns a route, the app renders pseudocode with the active line highlighted as the algorithm executes—like stepping through a debugger. You can watch Nearest Neighbor greedily pick the next closest stop, or watch 2-Opt try and reject segment reversals in real time.

5 What Makes This Interesting

The routing problem is the entry point, but the deeper question is: *which locations are actually worth visiting?* Route optimization assumes you already know where to go. The harder problem is figuring out where to go in the first place.

This leads to questions about:

Survival How long does a flyer last at a given location before it's removed or covered?

Exposure How many people actually see and engage with a board at that location?

Learning If you've never posted at a location before, how do you estimate its value from similar locations?

These questions are the subject of the companion paper, which develops a full Bayesian hierarchical framework combining survival analysis, spatial point processes, and multi-armed bandits. But the intuition is simple: a flyer on a busy transit shelter that gets removed in 6 hours is worth less than a flyer in a quiet laundromat that stays up for 3 months—even though the shelter has $100\times$ more foot traffic.

6 Current Status

As of May 2026: (1) 10 curated flyering locations across Cambridge (2) 13 flyers placed and photographed (3) An estimated 60 flyers in circulation city-wide (4) Three working solvers with step-by-step animation (5) A documented mathematical framework awaiting more observations

The project is intentionally small. It's a real task done by a real group, held to a standard of rigor that makes it useful as both a civic tool and a teaching artifact.

7 What's Next

(1) Real walking/biking distances via OSRM or GraphHopper (2) Population density and foot traffic heatmap overlay (3) Time-of-day simulation (morning commute vs. evening) (4) Confirmed vs. estimated placement visualization (5) Closing the loop: using collected evidence to update the predictive model

Links

(1) Application: <https://letters-cambridge.web.app/stories/flyering/>
(2) Organization: <https://www.cambridge-dev.org/>